

Summer 2026 Final Project Roadmap

Maze Exploration with Lily the Ant

ADT, Stack, and Queue Learning Experience

Project Goal

Build a console-based maze game in C where Lily the Ant explores a maze, collects points, and tries to achieve the highest score before time runs out. The project must also demonstrate program organization through an Abstract Data Type (ADT), a stack-based Undo feature, and a queue-based shortest-path search.

What Students Will Learn

- Represent a maze as an ADT with a clear public interface and private implementation details.
- Use a stack to store move history and undo the most recent move (LIFO behavior).
- Use a queue to explore maze positions in first-in, first-out order (FIFO behavior).
- Apply loops, arrays, strings, file I/O, functions, pointers, structures, and modular design in one project.
- Test normal cases, invalid input, missing files, boundaries, and empty-data-structure cases.

Important Clarification: What Is Required?

Required: Students must implement a Maze ADT, a Stack ADT for Undo, and a Queue ADT for a guided BFS shortest-path feature.

Not required: Students do not need to place every line of code in main.c, and they should not expose the internal arrays or indices of the ADTs to main.c.

Instructor support: Starter header files and function prototypes should be provided for the Stack and Queue so students focus on ADT behavior rather than guessing the interface.

Important Dates

Milestone	Date
Final Review Lecture	Wednesday, August 12, 2026
Final Exam Due	Thursday, August 13, 2026 at 8:00 PM ET
Final Project Due	Friday, August 14, 2026 at 9:30 AM ET

Required File Organization

```
project/  
├─ main.c  
├─ maze.c  
├─ maze.h  
├─ stack.c  
├─ stack.h  
├─ queue.c  
├─ queue.h  
└─ maze.txt
```

```
|— README.md
|— tests.md
```

main.c should control the game flow. The ADT files should contain the data and operations for the maze, stack, and queue.

Suggested ADT Interfaces

Maze ADT

```
int maze_load(Maze *maze, const char *filename);
void maze_print(const Maze *maze);
int maze_can_move(const Maze *maze, int row, int col);
int maze_move_lily(Maze *maze, char direction);
int maze_collect_point(Maze *maze);
```

Stack ADT — Move History and Undo

```
void stack_init(Stack *stack);
int stack_push(Stack *stack, Position value);
int stack_pop(Stack *stack, Position *value);
int stack_is_empty(const Stack *stack);
```

Every successful move pushes Lily's previous position onto the stack. When the user presses U, the program pops the most recent position and moves Lily back.

Queue ADT — Guided BFS Shortest Path

```
void queue_init(Queue *queue);
int queue_enqueue(Queue *queue, Position value);
int queue_dequeue(Queue *queue, Position *value);
int queue_is_empty(const Queue *queue);
```

When the user presses H, the program uses BFS and the Queue ADT to report the minimum number of steps from Lily's current position to the exit. Displaying the complete path may be an optional bonus.

Week-by-Week Project Plan

Week 1 — Project Setup and Design

- ☐ Create the GitHub repository and required folders/files.
- ☐ Create main.c and README.md.
- ☐ Draw a simple module diagram showing main, Maze ADT, Stack ADT, and Queue ADT.
- ☐ Write pseudocode for the game loop.

Deliverable: Repository, file structure, module diagram, and pseudocode.

Week 2 — Maze ADT

- ☐ Read the maze from maze.txt.

- ☐ Store the maze inside the Maze ADT.
- ☐ Display the maze through maze_print().
- ☐ Handle a missing or invalid maze file without crashing.
- ☐ Do not access the Maze ADT's internal array directly from main.c.

Deliverable: Maze ADT loads and displays valid files and safely rejects invalid files.

Week 3 — Lily Movement

- ☐ Place Lily in a valid open location.
- ☐ Move up, down, left, and right.
- ☐ Prevent movement through walls or outside the maze.
- ☐ Collect points and update the score.

Deliverable: Lily moves safely and scoring works.

Week 4 — Stack ADT: Undo

- ☐ Implement stack_init, stack_push, stack_pop, and stack_is_empty.
- ☐ Push Lily's previous position after each successful move.
- ☐ Add U to undo the most recent successful move.
- ☐ Handle Undo when the stack is empty.

Deliverable: The Undo feature correctly demonstrates LIFO behavior.

Week 5 — Queue ADT: BFS Hint

- ☐ Implement queue_init, queue_enqueue, queue_dequeue, and queue_is_empty.
- ☐ Use BFS to explore reachable cells from Lily's location.
- ☐ Add H to report the shortest number of steps to the exit.
- ☐ Report clearly when no path exists.

Deliverable: The Hint feature correctly demonstrates FIFO behavior and BFS.

Week 6 — Timer, Testing, and Polish

- ☐ Add the 60-second timer and final score.
- ☐ Use clear functions, comments, and variable names.
- ☐ Complete README build/run instructions and testing notes.
- ☐ Record a short demo video and push the final commit.

Deliverable: Complete, tested, documented project ready for grading.

Required Game Controls

Key	Action
W / A / S / D	Move Lily up, left, down, or right
U	Undo the most recent successful move using the Stack ADT
H	Run the Queue-based BFS hint and report shortest steps to the exit
Q	Quit and display the final score

Grading Rubric — 100 Points

Points	Requirement	Complete
10	Load and validate maze.txt	☐
8	Display the maze correctly	☐
8	Place and move Lily safely	☐
8	Prevent wall and boundary collisions	☐
8	Collect points and track score	☐
8	Timer, quit behavior, and final score	☐
12	Maze ADT design and encapsulation	☐
12	Stack ADT and Undo feature	☐
12	Queue ADT and BFS shortest-step hint	☐
6	Functions, headers, comments, and organization	☐
5	GitHub commits and repository quality	☐
3	README and testing notes	☐

Required Test Cases

#	Feature	Input / Setup	Expected Result	Pass
1	Valid maze file	Open a valid maze.txt	Maze loads and displays correctly	☐
2	Missing file	Run without maze.txt	Print an error and exit safely	☐
3	Movement	Use W/A/S/D	Lily moves only to valid cells	☐
4	Wall collision	Move toward a wall	Lily remains in the same position	☐
5	Point collection	Move onto a point	Score increases once	☐
6	Stack push	Make three valid moves	Three previous positions are stored	☐
7	Undo	Press U after moving	Lily returns to the previous position	☐
8	Empty Undo	Press U before any move	Display a safe message; do not crash	☐
9	Queue/BFS path	Use H when a path exists	Shortest step count is reported	☐
10	Queue/BFS no path	Use a blocked maze	Program reports that no path exists	☐
11	Invalid key	Press an unsupported key	Ignore it or display a message	☐
12	Quit	Press Q	Exit cleanly and show final score	☐

Optional Bonus Challenges

- Display the complete BFS shortest path, not only the number of steps (+3).
- Add a DFS exploration mode using a second stack (+3).
- Save and reload the highest score from a file (+2).
- Support multiple maze files selected from a menu (+2).

Clear Message for Students

This project is not only about making a maze game work. It is also about showing that you understand how an ADT hides data and exposes safe operations.

You must demonstrate all three ideas:

- Maze ADT: the maze is managed through functions declared in maze.h.
- Stack ADT: Undo removes the most recently saved position first.
- Queue ADT: BFS processes positions in the same order in which they were discovered.

A working game that keeps all data and logic inside main.c does not fully satisfy the ADT requirement.

Final Submission Checklist

- ☐ All .c and .h source files
- ☐ maze.txt test files, including at least one blocked maze
- ☐ README with build and run instructions
- ☐ tests.md with results for all required test cases
- ☐ Screenshots or a short demo video
- ☐ GitHub repository link
- ☐ Final commit pushed before the deadline